

From a naive sum to a Gaussian mixture

How Run A worked, and what changed in D+ — an intuitive walk-through

The shared skeleton

Both models encode an image into a 32×32 grid of latent vectors. At each location they build *two* things and add them:

$$z = \underbrace{z_{\text{vq}}}_{\text{a codeword } e_k} + \underbrace{z_{\text{cont}}}_{\text{a Gaussian offset}}, \quad z_{\text{cont}} = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

then decode $\hat{x} = \text{Dec}(z)$. The codebook picks the nearest of $K=256$ vectors e_k ; the continuous branch adds a normally-distributed detail. **Everything below is about *where the Gaussian is centered and what loss shapes it*.**

Run A — naive sum, standard prior ($\beta=0.001$)

Checkpoint `dualvae_20260724-121508_392b5b`. The continuous branch is a plain VAE with the textbook prior $\mathcal{N}(0, I)$, and the loss is pixel MSE plus a weak KL:

$$\mathcal{L}_A = \underbrace{\|x - \hat{x}\|^2}_{\text{reconstruction}} + \text{commit}(e_k) + \beta \underbrace{\text{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, I))}_{= \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2)}.$$

The intuition. The KL pulls the offset toward the *origin*: it drives $\mu \rightarrow 0$, *independently of the code* e_k . So z_{cont} is free to fill a whole unit-variance cloud, and because β is tiny the branch runs at high rate (near-deterministic, $\sigma \approx 0.02$) and simply *memorizes the image*. The codeword is just added on top:

$$z = e_k + z_{\text{cont}}, \quad \underbrace{\|z_{\text{cont}}\| \text{ (spread } \approx 1.0)}_{\text{does all the work}} \gg \underbrace{\text{codeword spacing } \approx 0.1}_{\text{tiny bumps}}.$$

The continuous cloud is $\sim 10\times$ wider than the gaps between codewords, so the codewords are invisible *dots inside one big blob*. The decoder learns to read z_{cont} and ignores the codes, which collapse to a near-constant offset ($\sim 64\%$ of z_{vq} is a shared DC vector, only $\sim 36\%$ real information).

$$p(z) \approx \text{one Gaussian} \quad (\text{separability } 0.11, \text{ no mixture}).$$

Result: excellent *reconstruction* (SSIM 0.882, PSNR 29.1) because the continuous branch is a high-rate near-deterministic code — but **no Gaussian mixture and non-semantic, barely-used codes**.

D+ — move the center to the code, then add realism

Checkpoint `dualvae_20260726-230704_866e33`. Two changes, one structural and one perceptual.

Change 1 (D): the code-centered prior — a single sign flip in where the Gaussian sits

Rewrite the continuous branch as a *residual* Δ around the code, $z = e_k + \Delta$, and move the prior's center from 0 onto the codeword:

$$p(\Delta | k) = \mathcal{N}(0, \sigma_k^2) \iff p(z | k) = \mathcal{N}(e_k, \sigma_k^2), \quad \text{KL} = \log \frac{\sigma_k}{\sigma} + \frac{\sigma^2 + \mu_\Delta^2}{2\sigma_k^2} - \frac{1}{2}.$$

The decisive term is $\mu_\Delta^2/(2\sigma_k^2)$: it penalizes the residual *mean*, so training now drives $\mu_\Delta \rightarrow 0$ — i.e. it pulls z **toward the codeword** e_k , not toward the origin. Averaging over how often each code is used (π_k) gives, *by construction*, a genuine mixture:

$$p(z) = \sum_k \pi_k \mathcal{N}(e_k, \sigma_k^2) \quad \text{well separated when } \sigma_k \ll \min_{j \neq k} \|e_j - e_k\|.$$

Now the roles *flip*: the codewords become the mixture means (they carry the image), and the continuous branch shrinks to a small within-component detail Δ . The single blob of Run A becomes tight clusters sitting on their codewords (**separability 0.11 $\rightarrow$ 3.0**), while reconstruction stays high because $z = e_k + \Delta$ still spans the same space. (Online, e_k is the EMA cluster mean and σ_k^2 the EMA within-code variance — code `residual_continuous+component_prior+use_ema_codebook`.)

Change 2 (+): perceptual + adversarial loss — optimize *realism*, not pixels

Add an LPIPS feature-space term and a PatchGAN discriminator D :

$$\mathcal{L}_{D+} = \underbrace{\|x - \hat{x}\|^2}_{\text{pixels}} + \lambda_p \underbrace{\|\phi(x) - \phi(\hat{x})\|^2}_{\text{LPIPS (perceptual)}} + \lambda_g \underbrace{\mathbb{E}[-\log D(\hat{x})]}_{\text{adversarial}} + \text{commit} + \beta \text{KL}_{\text{code-centered}}.$$

ϕ is a fixed feature network, so LPIPS matches images where the *eye* looks, and the GAN pushes $\hat{x}$ onto the manifold of real photos. This optimizes *perceptual* quality, so **rFID 22 $\rightarrow$ 4** and **LPIPS 0.12 $\rightarrow$ 0.04** while pixel PSNR/SSIM stay flat (the GAN trades pixel error for realism). It also drives the continuous branch to high rate (KL $\approx$ 2.6, $\sigma \approx$ 0.05) and *sharpens* each mixture component (separability $\rightarrow$ 3.0).

The one-line picture

Run A: $p(z | k) = \mathcal{N}(\underbrace{0}_{\text{origin}}, I) \Rightarrow$ codes drown, one blob, best pixels.

D+: $p(z | k) = \mathcal{N}(\underbrace{e_k}_{\text{codeword}}, \sigma_k^2) + \text{LPIPS/GAN} \Rightarrow$ genuine mixture, codes carry structure, photorealistic.

Moving the Gaussian's center from the origin to the codeword turns a single blob into a Gaussian mixture; the perceptual/adversarial loss then makes that mixture reconstruct photorealistically.

	prior center	separability	rFID / LPIPS
Run A ($\beta=.001$, std)	origin 0	0.11 (one blob)	24.4 / 0.11
D+ (code prior + LPIPS/GAN)	codeword e_k	3.0 (mixture)	3.97 / 0.04